

ITPROFEL2: Open Source Programming

Learning Module: Finals - Week 6

Instructor: Henry V. Dela Cruz | Topic: Objects and the 'self' Parameter

I. Overview and Objectives

A class is just a template. The true power of OOP is creating hundreds of distinct objects from that template, each holding its own unique data. This week, we explore how Python prevents data collisions between objects using the crucial `self` parameter.

Intended Learning Outcomes (ILOs):

- Create multiple distinct objects from a single class blueprint.
- Define what the `self` parameter represents in Python memory.
- Demonstrate how `self` establishes localized instance variables.
- Differentiate between Class Variables (shared) and Instance Variables (unique).

II. Understanding the 'self' Parameter

When you call a method on an object (e.g., `user1.login()`), Python secretly translates that into `UserClass.login(user1)`. It passes the object itself as the first argument. This is why class methods must always declare `self` as their first parameter. **`self` is the object looking at the mirror.**

III. Instance Variables via 'self'

By attaching variables to `self` (e.g., `self.status = "Active"`), you create an **Instance Variable**. Unlike Class Variables (which are shared by all), Instance Variables belong strictly to the object that created them. This isolates data.

```
class Device:
    manufacturer = "TechCorp" # Shared Class Variable

    def set_ip(self, ip_address):
        # Unique Instance Variable
        self.ip = ip_address

phone = Device()
laptop = Device()

phone.set_ip("192.168.1.10")
laptop.set_ip("10.0.0.5")

print(phone.ip)    # Output: 192.168.1.10
print(laptop.ip)   # Output: 10.0.0.5 (Completely isolated from phone!)
```

IV. Memory and Multiple Objects

Because of `self`, phone and laptop can exist side-by-side in memory without their IP addresses overwriting each other. They share the same methods and class variables, but maintain their own distinct state maps.

Laboratory Session: Multi-Entity Interaction

Scenario: You must prove that your `UserEntity` blueprint can handle multiple users simultaneously without their data colliding. You will create a method that assigns a unique permission role to an entity, securely attached to the object's memory via `self`.

Instructions:

1. Use your `UserEntity` class (keep the institution class variable).
2. Create a new method inside the class called `assign_role(self, new_role)`.
3. Inside that method, map the parameter `new_role` to the object's instance variable: `self.role = new_role`.
4. Outside the class, instantiate **two separate objects**: `user1 = UserEntity()` and `user2 = UserEntity()`.
5. Call `assign_role("Administrator")` on `user1`.
6. Call `assign_role("Guest")` on `user2`.
7. Print both users' `.role` attributes to prove they isolated their states correctly.

Expected Terminal Output:

```
--- INDEPENDENT ENTITY VERIFICATION ---  
User 1 Role: Administrator  
User 2 Role: Guest  
Verification: Objects are maintaining isolated states.
```

Submit your .py file. Your entity framework is now capable of handling unique data parameters.